

# Lab: File Path Traversal, Traversal Sequences Blocked with Absolute Path Bypass

PortSwigger Web Security Academy — Parcial I: Fundamentos de Ciberseguridad

## 1. Nombre del laboratorio

File path traversal, traversal sequences blocked with absolute path bypass

**Fuente oficial:**

<https://portswigger.net/web-security/file-path-traversal/lab-absolute-path-bypass>

## 2. Nivel

Practitioner.

## 3. Tipo de explotación

Path Traversal / Directory Traversal (CWE-22), con bypass de un filtro de secuencias de traversal (blacklist de "../") mediante el uso de una ruta absoluta del sistema de archivos.

## 4. Objetivo del laboratorio

La aplicación vulnerable es la misma tienda en línea que sirve imágenes de producto a través de GET /image?filename=NOMBRE. A diferencia del caso simple, aquí el servidor sí bloquea activamente las secuencias de traversal ("../") en el parámetro filename. El objetivo es identificar esta defensa parcial y evadirla para leer el contenido del archivo /etc/passwd del sistema.

## 5. Explicación técnica de la vulnerabilidad

### 5.1 Concepto y mecanismo del bypass

El servidor implementa una defensa básica contra path traversal: antes de usar el parámetro filename, revisa (y bloquea) la presencia de secuencias como "../". Sin embargo, esta validación solo cubre las rutas relativas; el servidor sigue tratando el valor de filename como una ruta relativa a un directorio de trabajo por defecto (por ejemplo, /var/www/images/), pero no verifica si dicho valor es en realidad una ruta absoluta.

Cuando una aplicación construye la ruta final usando una función de unión de rutas del sistema operativo o del lenguaje (equivalente a path.join(base\_dir, filename)), muchas de estas funciones tienen un comportamiento especial y poco conocido: si el segundo argumento (filename) es una ruta absoluta (comienza con "/" en Unix), el resultado de la unión es esa ruta absoluta, descartando por completo el directorio base. Esto ocurre, por ejemplo, en os.path.join de Python y en funciones equivalentes de otros lenguajes/entornos.

Por lo tanto, aunque el filtro de la aplicación bloquee con éxito cualquier valor que contenga "../", nunca llega a evaluar el caso en que filename es directamente /etc/passwd: no hay secuencia de traversal que bloquear, y la función de unión de rutas entrega el archivo absoluto solicitado en lugar de buscarlo dentro de /var/www/images/.

### 5.2 Por qué el traversal simple falla aquí

Al enviar el payload `../../../../etc/passwd` (idéntico al usado en el laboratorio "simple case"), el servidor responde con 400 Bad Request y el mensaje "No such file", confirmando que el filtro de secuencias de traversal está activo y detecta la subcadena `"../"` antes de intentar acceder al archivo. Este es precisamente el obstáculo que distingue a este laboratorio del anterior: la defensa funciona correctamente contra el ataque más obvio, pero deja abierta la variante de ruta absoluta.

### 5.3 Impacto y clasificación estándar

El impacto es idéntico al del caso simple: exposición de información sensible del sistema operativo (confidencialidad), clasificada como CWE-22 (Improper Limitation of a Pathname to a Restricted Directory) y encuadrada en OWASP Top 10 2021 — A01:2021 Broken Access Control. La diferencia relevante está en la causa raíz: aquí no es la ausencia total de validación, sino una validación incompleta que cubre un solo vector (secuencias relativas) y omite otro (rutas absolutas), un patrón de fallo muy común en defensas basadas en listas negras (blacklist) en lugar de listas blancas (allow-list).

La defensa efectiva combina no confiar en ninguna forma de la entrada del usuario para construir rutas de archivo, canonicalizar y validar la ruta resultante contra el directorio base permitido —incluyendo explícitamente el caso de rutas absolutas—, y aplicar el principio de menor privilegio como capa adicional; estos puntos se desarrollan en la sección 8 (Mitigación recomendada).

## 6. Procedimiento paso a paso

### Paso 1 — Reconocimiento de la aplicación

Se navega por la tienda en línea y se identifica nuevamente la funcionalidad de imágenes de producto servidas mediante `GET /image?filename=NOMBRE`.

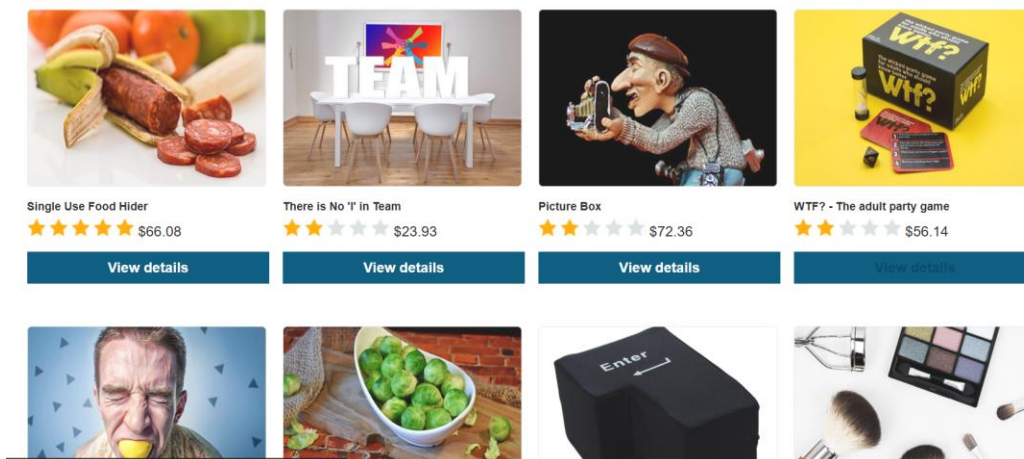


Figura 1. Catálogo de productos de la aplicación vulnerable.

### Paso 2 — Interceptación del tráfico con Burp Suite

Se activa el interceptor de Burp Suite y se recarga la página principal para capturar la solicitud inicial y confirmar que el proxy está interceptando correctamente el tráfico del navegador.

| Time              | Type | Direction | Method | URL                                                                |
|-------------------|------|-----------|--------|--------------------------------------------------------------------|
| 15:11:36 22 Se... | HTTP | → Request | GET    | https://0a350056037ca9d8832ce20300c000d6.web-security-academy.net/ |

Request

Pretty

Raw

Hex

```

1 GET / HTTP/2
2 Host: 0a350056037ca9d8832ce20300c000d6.web-security-academy.net
3 Cookie: session=6TpHJAwB85sPvcK2HfxglvK528iXpukq
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="55"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: es-419,es;q=0.9
9 Upgrade-Insecure-Requests: 1
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36

```

Figura 2. Solicitud GET / interceptada en Burp Suite al recargar la página.

### Paso 3 — Identificación del endpoint vulnerable

En el historial de peticiones (Proxy > HTTP history) se localizan las solicitudes GET /image?filename=X.jpg correspondientes a las imágenes de los productos, confirmando el parámetro controlable por el usuario.

| Time              | Type | Direction | Method | URL                                                                                     |
|-------------------|------|-----------|--------|-----------------------------------------------------------------------------------------|
| 15:12:31 22 Se... | HTTP | → Request | GET    | https://0a350056037ca9d8832ce20300c000d6.web-security-academy.net/image?filename=20.jpg |
| 15:12:31 22 Se... | HTTP | → Request | GET    | https://0a350056037ca9d8832ce20300c000d6.web-security-academy.net/image?filename=58.jpg |
| 15:12:31 22 Se... | HTTP | → Request | GET    | https://0a350056037ca9d8832ce20300c000d6.web-security-academy.net/image?filename=7.jpg  |
| 15:12:31 22 Se... | HTTP | → Request | GET    | https://0a350056037ca9d8832ce20300c000d6.web-security-academy.net/image?filename=17.jpg |
| 15:12:31 22 Se... | HTTP | → Request | GET    | https://0a350056037ca9d8832ce20300c000d6.web-security-academy.net/image?filename=55.jpg |
| 15:12:31 22 Se... | HTTP | → Request | GET    | https://0a350056037ca9d8832ce20300c000d6.web-security-academy.net/image?filename=23.jpg |
| 15:12:31 22 Se... | HTTP | → Request | GET    | https://0a350056037ca9d8832ce20300c000d6.web-security-academy.net/image?filename=37.jpg |
| 15:12:31 22 Se... | HTTP | → Request | GET    | https://0a350056037ca9d8832ce20300c000d6.web-security-academy.net/image?filename=51.jpg |
| 15:12:32 22 Se... | HTTP | → Request | GET    | https://0a350056037ca9d8832ce20300c000d6.web-security-academy.net/academyLabHeader      |

Request

Pretty

Raw

Hex

```

1 GET /image?filename=20.jpg HTTP/2
2 Host: 0a350056037ca9d8832ce20300c000d6.web-security-academy.net
3 Cookie: session=6TpHJAwB85sPvcK2HfxglvK528iXpukq
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: es-419,es;q=0.9
6 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="55"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8

```

Figura 3. Historial de peticiones mostrando solicitudes al endpoint /image?filename=.

### Paso 4 — Envío de la solicitud a Repeater

Se selecciona una de las solicitudes de imagen y, mediante el menú contextual, se envía al módulo Repeater para manipular el parámetro filename de forma controlada.

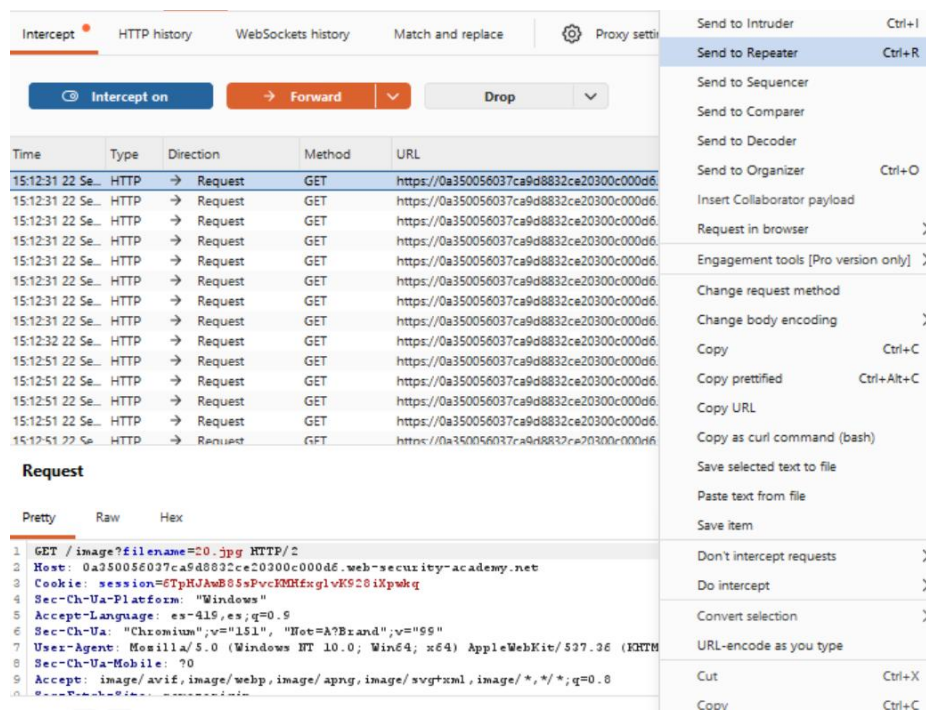


Figura 4. Envío de la solicitud interceptada al módulo Repeater ("Send to Repeater").

## Paso 5 — Prueba de traversal simple (bloqueada)

Se prueba primero el payload clásico `../../../../etc/passwd`. El servidor responde con 400 Bad Request y el mensaje "No such file", confirmando que existe un filtro que bloquea las secuencias de traversal relativas.



Figura 5. Payload `../../../../etc/passwd` bloqueado por el servidor (400 Bad Request, "No such file").

## Paso 6 — Bypass mediante ruta absoluta

Se sustituye el valor del parámetro filename por la ruta absoluta `/etc/passwd`, sin ninguna secuencia de traversal. El servidor responde con 200 OK y devuelve el contenido íntegro del archivo, confirmando el bypass del filtro.

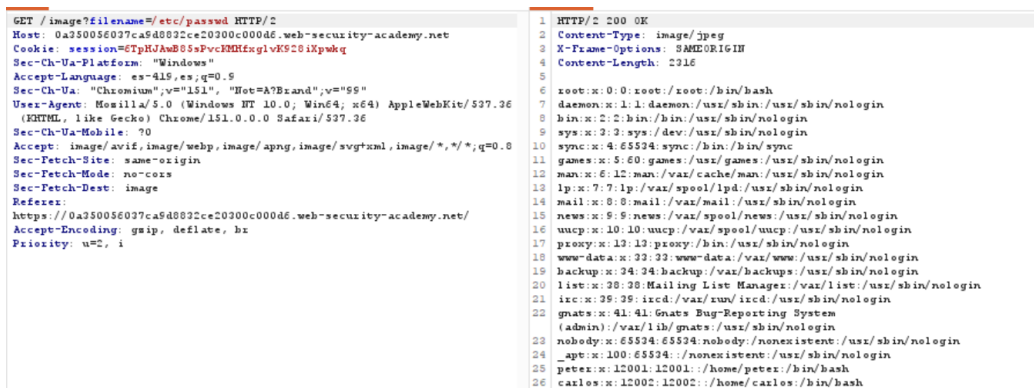


Figura 6. Payload `/etc/passwd` (ruta absoluta) — respuesta 200 OK con el contenido del archivo.

## Paso 7 — Confirmación de resolución del laboratorio

PortSwigger Web Security Academy confirma la explotación exitosa marcando el laboratorio como "Solved".

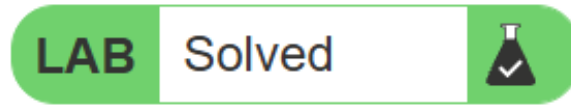


Figura 7. Confirmación "Lab Solved" en PortSwigger Web Security Academy.

## 7. Explicación del payload

**Payload bloqueado:**

`../../../../etc/passwd`

**Payload exitoso:**

`/etc/passwd`

Funcionamiento técnico:

- El primer payload (`../../../../etc/passwd`) es detectado por el filtro de la aplicación, que busca activamente la subcadena `"../"` en el valor de filename y rechaza la solicitud con 400 Bad Request antes de tocar el sistema de archivos.
- El segundo payload (`/etc/passwd`) no contiene ninguna secuencia de traversal, por lo que pasa el filtro sin ser detectado.
- Al construir la ruta final, el servidor combina su directorio base (por ejemplo, `/var/www/images/`) con el valor de filename mediante una función de unión de rutas. Como filename ya es una ruta absoluta (comienza con `"/"`), esa función descarta el directorio base y devuelve directamente `/etc/passwd`.
- El sistema de archivos localiza y lee `/etc/passwd` sin ninguna restricción adicional, y el servidor lo devuelve en el cuerpo de la respuesta HTTP como si fuera una imagen más.

En esencia, el bypass no rompe el filtro de traversal: lo evita por completo, aprovechando un caso que el filtro nunca fue diseñado para cubrir. Esto ilustra un principio central en seguridad de aplicaciones: una defensa que bloquea un patrón específico (`"../"`) no equivale a validar que la entrada completa sea segura.

## 8. Mitigación recomendada

- No basar la validación únicamente en el bloqueo de secuencias de traversal (`"../"`); rechazar explícitamente cualquier valor de filename que comience con `"/"` (o con una letra de unidad en Windows, como `"C:\"`), es decir, cualquier ruta absoluta.
- Preferir un identificador indirecto (un ID numérico o nombre interno) que el servidor traduzca a la ruta real mediante un mapeo controlado, en lugar de usar directamente la entrada del usuario para construir rutas de archivo.
- Validar la entrada contra una lista blanca (allow-list) de nombres o extensiones permitidos, en lugar de listas negras que bloquean patrones específicos y son fáciles de evadir.
- Canonicalizar la ruta resultante (resolver `"../"`, enlaces simbólicos y rutas absolutas) y verificar de forma explícita que la ruta canonicalizada permanezca dentro del directorio base permitido, sin confiar en el comportamiento por defecto de las funciones de unión de rutas del lenguaje.

- Aplicar el principio de menor privilegio: ejecutar el proceso del servidor con un usuario restringido sin permisos de lectura sobre archivos sensibles del sistema operativo.
- Aislar el directorio de archivos servidos mediante chroot jails, contenedores o sistemas de archivos de solo lectura para el contenido público.
- Registrar y monitorear solicitudes con patrones anómalos en parámetros de archivo (rutas absolutas, "../", codificaciones) para detectar intentos de explotación.